

CS 301 · MEETING 11 · ACT 0

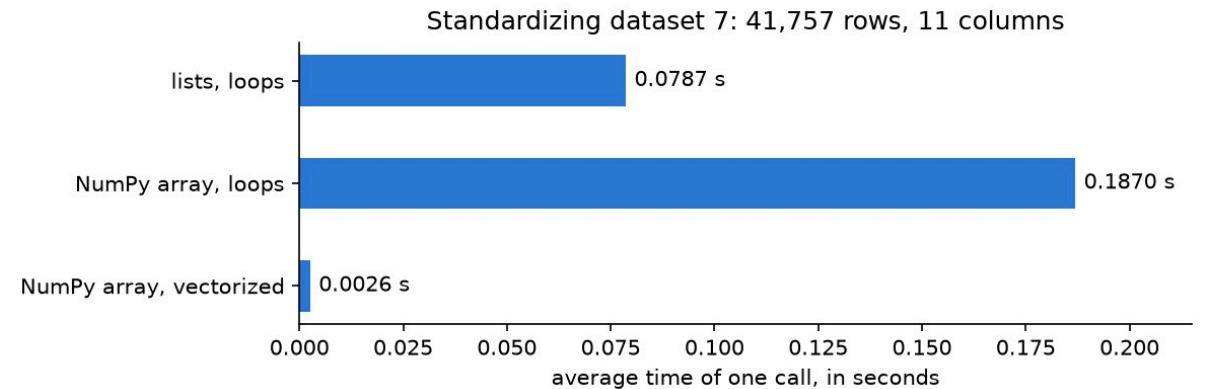
# Meeting 10, Revisited

Six ideas of meeting 10 that this meeting builds on: vectorization, broadcasting, slicing, axes, word vectors, and batches.

# Vectorization: a calculation without a loop

```
def standardize_array_loops(X):
    n, m = X.shape
    Z = np.empty((n, m))
    for j in range(m):
        total = 0.0
        for i in range(n):
            total += X[i, j]
        mean = total / n
        total = 0.0
        for i in range(n):
            total += (X[i, j] - mean) ** 2
        std = math.sqrt(total / n)
        for i in range(n):
            Z[i, j] = (X[i, j] - mean) / std
    return Z
```

```
def standardize_vectorized(X):
    return (X - X.mean(axis=0)) / X.std(axis=0)
```



The same arithmetic. In the one-line version the loop over the numbers runs inside NumPy, in compiled code: tens of times faster than the loops in Python, and a Python loop over a NumPy array is the slowest of all.

# Broadcasting: a shape that mathematics would refuse

```
mu = X.mean(axis=0)           # one mean for each column: m numbers
sigma = X.std(axis=0)
Z = (X - mu) / sigma          # standardization, in one line
print("shape of X:", X.shape, " shape of mu:", mu.shape, " shape of X - mu:", (X - mu).shape)
v = np.array([1.0, 2.0, 3.0, 4.0])
print("v + 10 =", v + 10, " one number, added to every entry")
```

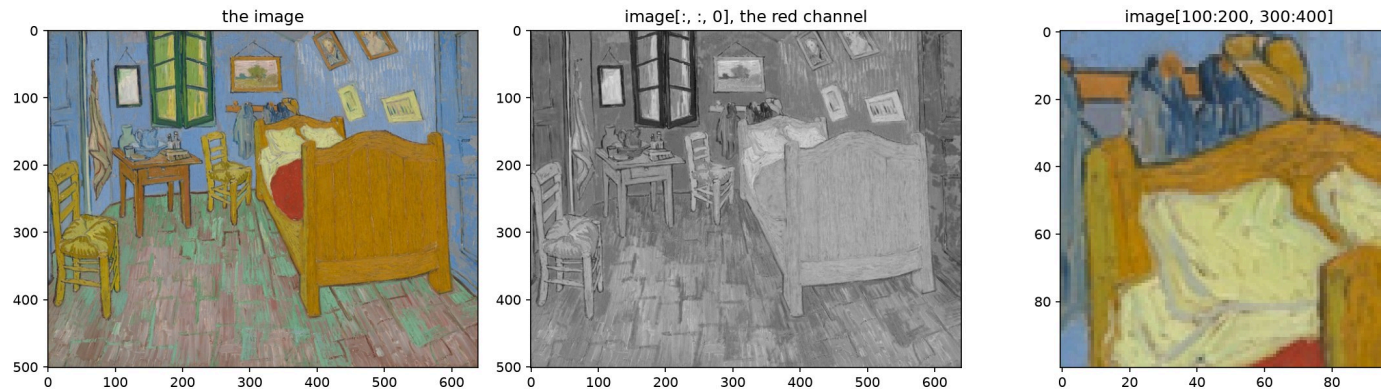
```
shape of X: (41757, 11)  shape of mu: (11,)  shape of X - mu: (41757, 11)
v + 10 = [11. 12. 13. 14.]  one number, added to every entry
```

- `X - mu`:  $(n, m)$  minus  $(m,)$ . In mathematics the shapes do not fit: the  $m$  means would have to be written on every row first,  $\mathbf{1}\mu^T$ . NumPy does that for you, without storing  $n$  copies: **broadcasting**.
- The rule: shapes are compared from the right, and a dimension that is missing, or is 1, is repeated. `v + 10` repeats one number for every entry. Today the bias of a layer is added in the same way.

# Slicing: a row, a column, a channel, a block

```
print("X[0]                a row, one point:          shape", X[0].shape)
print("X[:, 0]             a column, one feature:     shape", X[:, 0].shape)
print("image               H rows, W columns, 3 channels:", image.shape)
print("image[:, :, 0]       the red channel:          shape", image[:, :, 0].shape)
print("image[100:200, 300:400] a block of pixels:      shape", image[100:200, 300:400].shape)
```

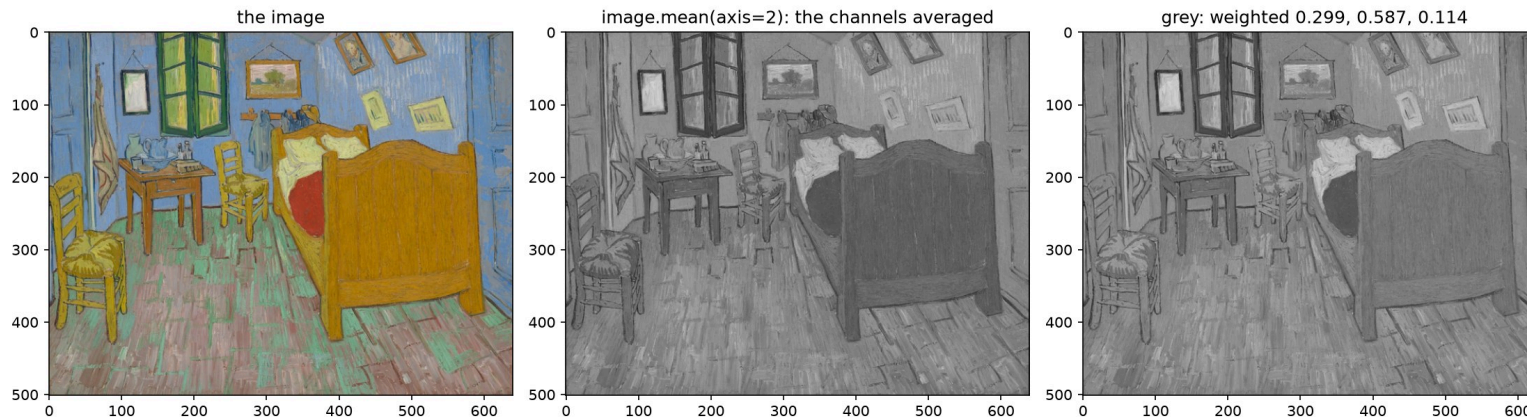
```
X[0]                a row, one point:          shape (11,)
X[:, 0]             a column, one feature:     shape (41757,)
image               H rows, W columns, 3 channels: (501, 640, 3)
image[:, :, 0]       the red channel:          shape (501, 640)
image[100:200, 300:400] a block of pixels:      shape (100, 100, 3)
```



# Along an axis: the dimension that disappears

```
print("X.mean(axis=0)      down the rows: one number for each column  shape", X.mean(axis=0).shape)
print("X.mean(axis=1)      along the columns: one number for each row  shape", X.mean(axis=1).shape)
print("image.mean(axis=2)   over the three channels of each pixel      shape", image.mean(axis=2).shape)
```

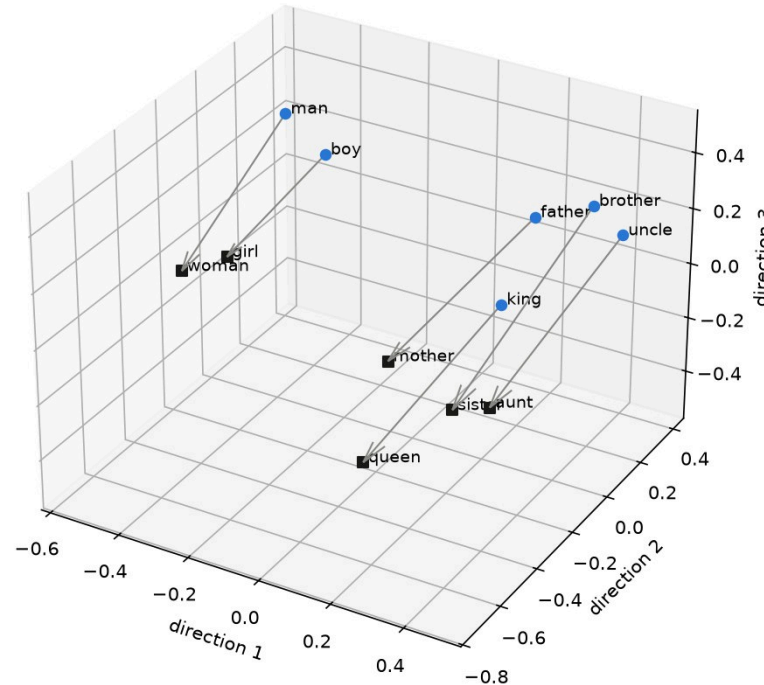
|                                 |                                            |                  |
|---------------------------------|--------------------------------------------|------------------|
| <code>X.mean(axis=0)</code>     | down the rows: one number for each column  | shape (11,)      |
| <code>X.mean(axis=1)</code>     | along the columns: one number for each row | shape (41757,)   |
| <code>image.mean(axis=2)</code> | over the three channels of each pixel      | shape (501, 640) |



The axis that is named is the dimension that disappears; the other dimensions keep their places.  
Today `S.max(axis=1)` takes the largest entry of every row.

# Word vectors: arrows in a space of 300 dimensions

12 word vectors of 300 numbers, drawn in three dimensions



vector of king - man + woman, nearest words: ['queen', 'princess', 'throne']

A word is a vector of 300 numbers; the difference of two words is a direction. The picture keeps three of the 300 dimensions, so it is a shadow of the arrows. `U @ v` compared one word with all 50,000 in one product.

# One point, then a batch

| data              | one point     | a batch of points |
|-------------------|---------------|-------------------|
| a day of weather  | (2,)          | (6940, 2)         |
| a colour image    | (501, 640, 3) | (30, 96, 96, 3)   |
| a word            | (300,)        | (50000, 300)      |
| a text of L words | (5, 300)      | (3, 9, 300)       |

- A **batch** adds one dimension at the front: the number of the point. The dimensions after it are the point's own:  $m$  for a day,  $H \times W \times 3$  for an image,  $L \times m$  for a text, padded with rows of zeros to one length.
- Every operation of meeting 10 ran on a whole batch at once:  $X - \mu$  on all the rows, the grey rule on thirty images,  $U @ v$  on 50,000 words.

# What this meeting adds

Meeting 10 stored **data** as arrays. Today a **rule** that reads a point, a **neuron**, is stored the same way: the rules of a layer are the rows of a matrix  $W$ , and applying them to a whole batch is one product.

- **Act 1:** a layer of four neurons on days of weather at Newark airport, then five neurons on the pixels of your image, in NumPy.
- **Act 2:** the same layer in PyTorch, and on a GPU.
- What a product costs waits for meeting 12.
- **Ask Colab for a GPU before you run anything** in the class notebook: it says how. Colab erases the session when its runtime type changes.